# 具身智能基础及实践

智能技术学部

2026年3月

# 课程安排

| 时间安排 | 教学内容 | 内容负责人 |
| --- | --- | --- |
| 第1周 | 具身智能概述 | 裴颂文 |
| 第2周 | 机器人计算系统<br>自主机器人的感知技术 | 张裕、宗鸣 |
| 第3周 | 计算机系统与嵌入式系统开发<br>自主机器人的规划与控制系统 | 潘旺 |
| 第4周 | 感觉运动系统<br>定位与导航 | 王鹤霖、杨博弘 |
| 第5周 | 课程考核 | 全体 |

# 第2章-机器人计算系统

张 裕

智能技术学部人工智能专业

# 目录


2.1 概述

2.2 自主机器人计算系统

2.3 自动驾驶

# 目录

2.4 具身智能机器人

2.5 ROS原理与应用

# 前 言

第一节课同学们已经了解了具身智能的内涵、组成及现状与挑战，本节课开始带领大家具体学习具身智能机器人各个基础模块，让大家对具身智能有更加深入的认知及为后续的实操打下基础。

# 2.1 概述

通过上一节课的学习，请同学思考一下，发展具身智能机器人的目的是什么?

- 解决复杂现实世界的任务

家庭与服务/工业与制造/医疗

- 实现真正意义上的通用人工智能路径
- 提升人机协作的自然性和效率
- 创造新的经济和社会价值

讨论：具身智能机器人能够解决复杂任务的关键组成部分是什么?

计算系统/大脑

# 2.2 自主机器人计算系统

■ 计算系统是自主机器人的关键部件，实现与环境交互和自主地完成任务；

■ 计算这类系统一般由传感器、感知、决策和执行模块组成数据处理流水线，完成从传感器数据中提取物理世界信息、规划任务和按规划控制机器人的计算任务；

举例：家庭扫地机器人

（1）传感器：激光雷达、摄像头、陀螺仪与里程计等；

（2）感知：扫地机器人需要在未知的家庭环境中移动，首先得知道自己在哪里、周围有什么；同步定位与地图构建：它如何记住环境；

（3）决策规划：它如何决定去哪里，有了地图后，机器人需要决定“先扫哪里，怎么扫”。

（4）执行：这是物理执行层，将规划转化为实际清洁动作。

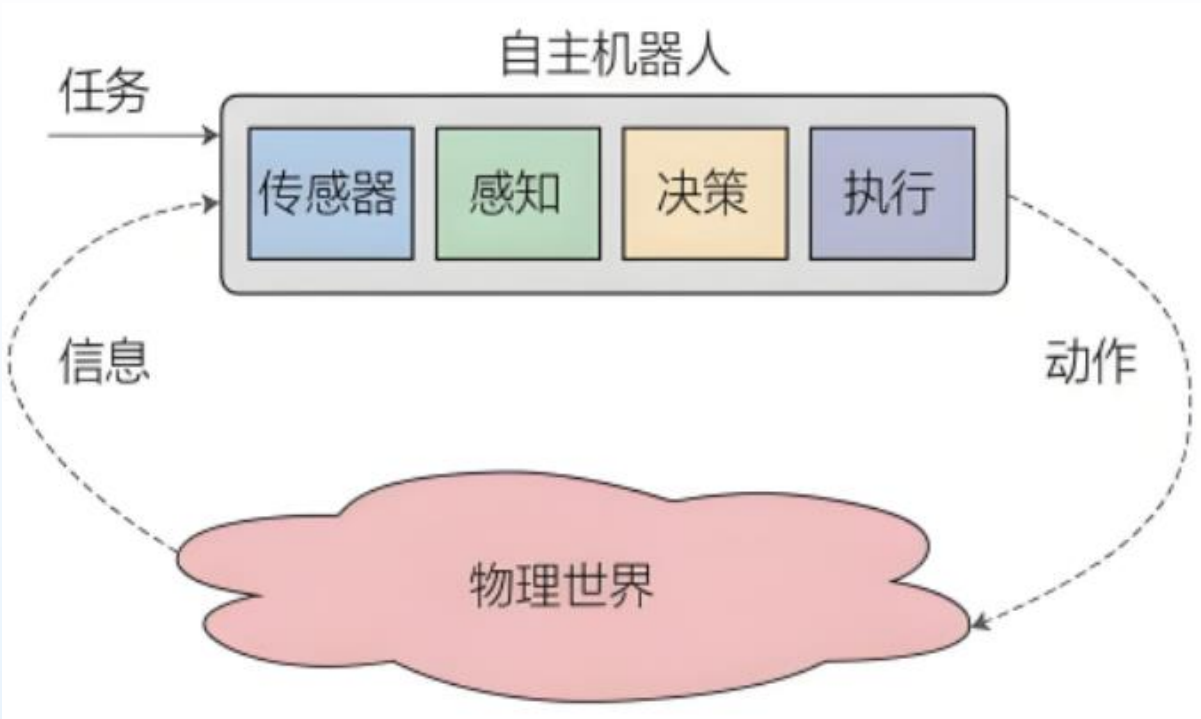



图2.1 自主机器人与环境的交互模型

# 2.2 自主机器人计算系统

传感器数据 → 感知 & SLAM → 世界模型 → 任务/运动规划 → 控制指令 → 执行机构

## 硬件平台

- 传感器组
  摄像头 / 激光雷达 / IMU / GPS / 毫米波雷达
- 异构计算单元
  CPU / GPU / TPU / FPGA / MCU
- 执行机构
  电机 / 液压 / 机械臂关节 / 驱动器
- 机械本体 / 底盘
  结构框架 / 轮式 / 履带 / 足式

## 感知层

- 原始数据预处理
  滤波 / 同步 / 点云分割
- 定位与建图 (SLAM)
  激光SLAM / V-SLAM / 多传感器融合
- 目标检测与跟踪
  障碍物识别 / 行人轨迹预测
- 本体状态估计
  里程计 / IMU融合 / 运动学

## 决策规划层

- 世界模型 / 场景理解
  语义地图 / 占据栅格 / 时空融合
- 任务级规划
  大语言模型(LLM) / 行为树 / 状态机
- 运动规划
  全局路径规划 (A*/RRT*) / 局部轨迹优化
- 行为决策
  避障策略 / 交互博弈 / 优先级仲裁

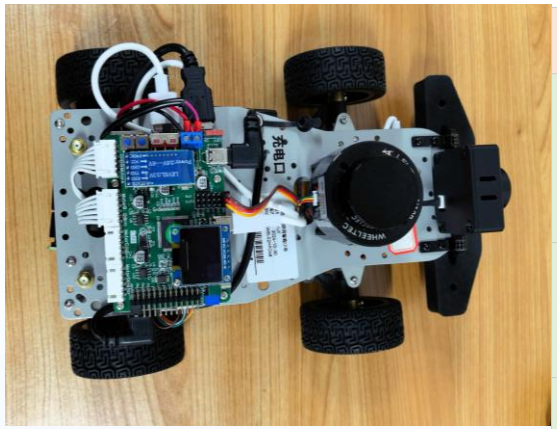
## 控制执行层

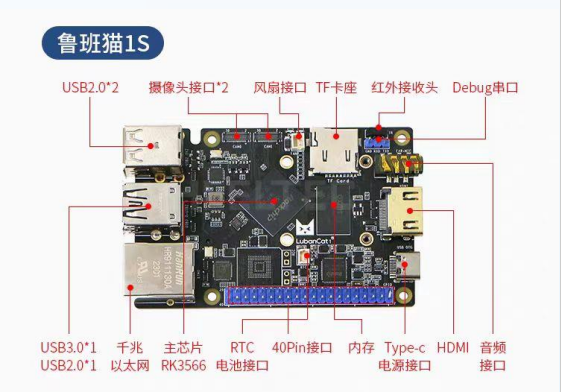
- 运动控制
  PID / 模型预测控制 (MPC) / 鲁棒控制
- 关节 / 电机伺服
  力矩 / 速度 / 位置环
- 安全监控与故障保护
  看门狗 / 碰撞检测 / 紧急制动 / 降级策略

## 支撑系统

- 通信中间件
  ROS 2 / DDS / ZeroMQ
- 仿真环境 / 数字孪生
  Gazebo / Isaac Sim / 硬件在环
- 参数配置与状态机
  动态参数服务器 / 生命周期管理
- 日志 & 调试系统
  录包 / 可视化 / 遥操作

# 2.2 自主机器人计算系统






| 模块 | 核心作用 |
|---|---|
| 树莓派(SD卡) | 中央控制器，负责处理传感器数据、执行导航算法，感知数据处理（摄像头、激光雷达）、运动决策 / 路径规划、下发高层运动指令（如 “前进 0.5m”） |
| STM32控制板 | 接收 ROS 指令、解析为电机控制信号（PWM）、驱动电机 / 舵机执行、反馈执行状态 |
| 摄像头 | 环境感知（如识别道路、障碍物、目标点），输出图像 / 特征数据 |
| 激光雷达 | 距离感知（障碍物精准测距、避障、建图），输出点云数据 |
| 电机 / 舵机 | 执行机构：直流电机控制车轮转速（前进 / 后退），舵机控制车轮转向（左 / 右） |
| ROS | 作为机器人的操作系统，提供了丰富的库和工具，便于编写、调试和部署机器人的应用程序。 |

## ■ 智能小车软硬件关系

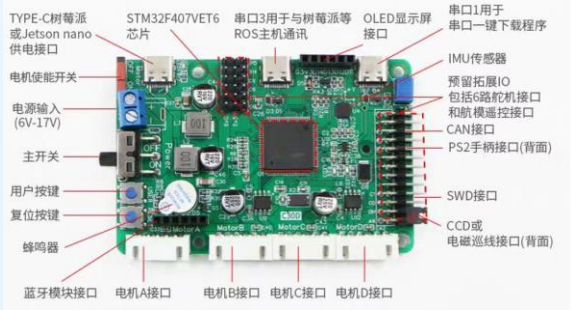
树莓派=主机（运行 Linux，跑 ROS，处理摄像头、雷达数据、决策规划）

SD 卡 = 树莓派的 “硬盘”

Linux = 装在硬盘里的操作系统

ROS = 大脑（装在 Linux 里的软件）

STM32=控制手脚（接收 ROS指令，驱动电机、舵机真正动起来）



# 2.3 自动驾驶

## 2.3.1 自动驾驶概述

- 自动驾驶汽车（Autonomous vehicles；Self-driving automobile ）又称无人驾驶汽车、电脑驾驶汽车、或轮式移动机器人，是一种通过电脑系统实现无人驾驶的智能汽车。在20世纪已有数十年的历史，21世纪初呈现出接近实用化的趋势。
- 自动驾驶是对社会影响最大的机器人应用之一（缓解驾驶疲劳/提升驾驶安全/降低交通事故发生率)
- 自动驾驶汽车由于配备了众多传感器，并对安全性有极高的要求，**自动驾驶计算系统也成为最复杂的机器人计算系统之一**

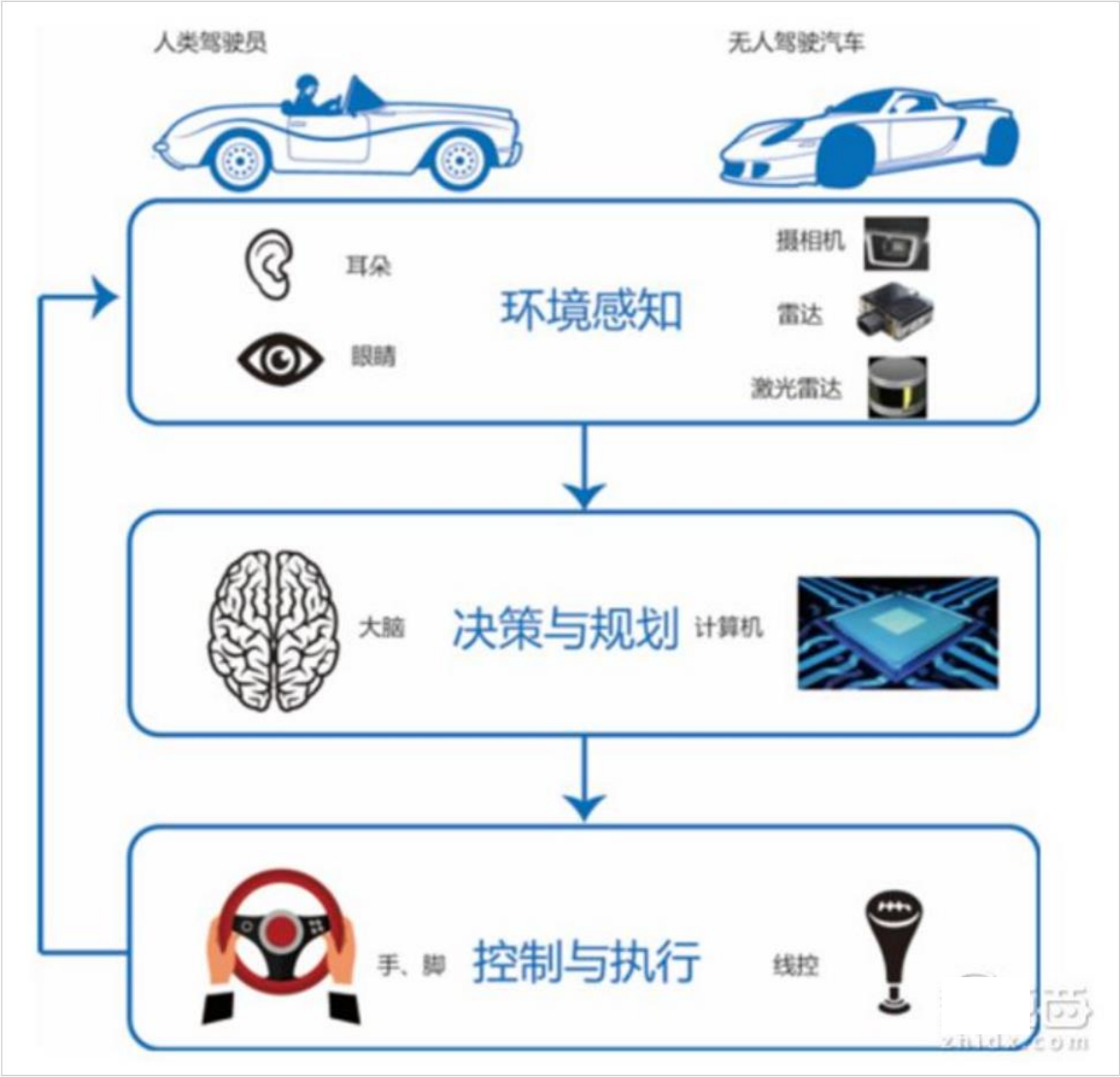
# 2.3 自动驾驶

## 2.3.1 自动驾驶概述

| 辅助驾驶硬件 | |
|---|---|
| 驾驶辅助级别 | • L2 |
| 辅助驾驶系统 | • HUAWEI ADS |
| 激光雷达品牌 | • HUAWEI华为 |
| 激光雷达数量 | • 1个 |
| 激光雷达线数 | • 192线 |
| 激光雷达10%反射率探测距离[m] | • 250 |
| 毫米波雷达 | • 3个 |
| 超声波雷达 | • 前6个/后6个 |
| 前方感知摄像头类型 | • 双目 |
| 前方感知摄像头像素[万] | • 800 |
| 环境感知摄像头 | • 7个 |
| 环视摄像头 | • 4个 |
| 环视摄像头像素[万] | • 250 |
| 车内摄像头 | • 1个 |

# 2.3 自动驾驶

## 2.3.2 自动驾驶计算系统



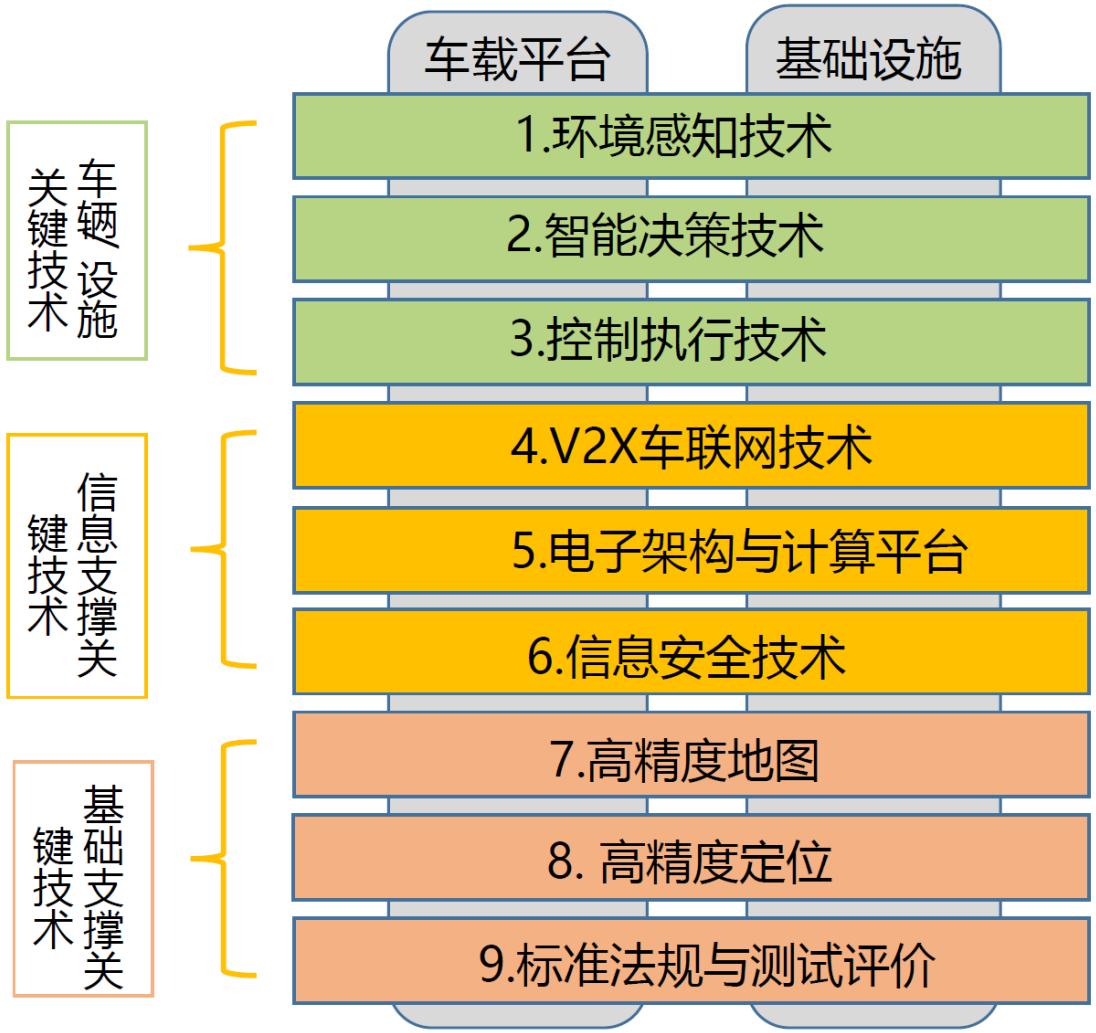
# 2.3 自动驾驶

## 2.3.2 自动驾驶计算系统

1. 计算系统的传感
2. 计算系统的感知
3. 计算系统的定位
4. 计算系统的规划
5. 计算系统的控制

# 2.3 自动驾驶

## 2.3.2 自动驾驶计算系统




### 愿 景

- **安全**
  大幅降低交通事故和交通事故伤亡人数
- **效率**
  显著提升交通效率
- **节能减排**
  有效降低交通能源消耗和污染排放
- **舒适和便捷**
  提高驾驶舒适性，解放驾驶员
- **人性化**
  使老年人、残疾人等都拥有驾车出行的权利

# 2.3 自动驾驶

## 2.3.3 端到端自动驾驶

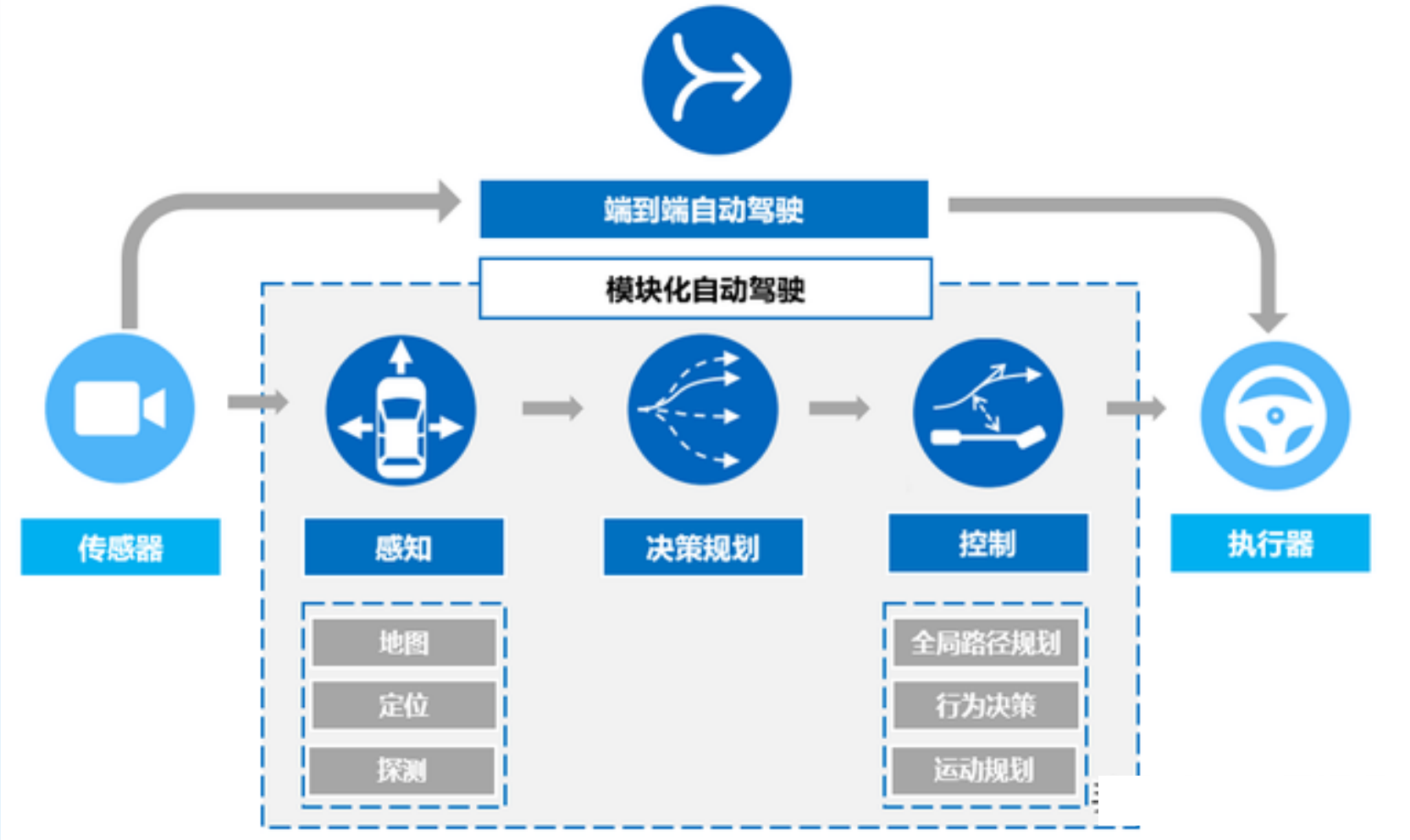
2025年1 月 16 日，理想的 OTA7.0 正式版开启了全量车主推送。随着 OTA7.0 的全量推送，理想宣布，其 OneModel 端到端 + VLM 智驾系统已经实现了「全场景端到端」能力。

与之同时上线的还有行业首创的 AI 推理可视化能力，它首次将自动驾驶过程中 AI 模型思考推理的过程可视化展现出来，让我们第一次看到了 AI 的思考和执行过程，因此也可以更加放心。同时，理想也成为了国内唯一、全球第二家将端到端技术应用到城市、高速及环路 NOA 场景的车企。

# 2.3 自动驾驶

## 2.3.3 端到端自动驾驶

**自动驾驶系统没有严谨的分类，但行业内普遍将自动驾驶系统区别为模块化的和端到端的。**



# 2.4 具身智能机器人

■ 现代机器人技术已成功应用于工业产线自动化、家庭服务和交通出行等领域，能够出色地完成这些领域的具体任务，然而它们只能在这些限定的场景下完成预先设定的任务，一旦场景发生变化或任务需要微调，机器人计算系统便要重新编程和调整。

■ 近年来，Transformer架构和生成式人工智能技术取得了突破，大语言模型和多模态模型展现了与人类相似的语言理解、表达和逻辑推理能力，强化学习技术也展示了机器人从环境中学习完成复杂控制任务的能力，这些进展让机器人科学家和工程师看到了实现具身智能机器人的可能性。

## 2.4.1 从自动驾驶到具身智能

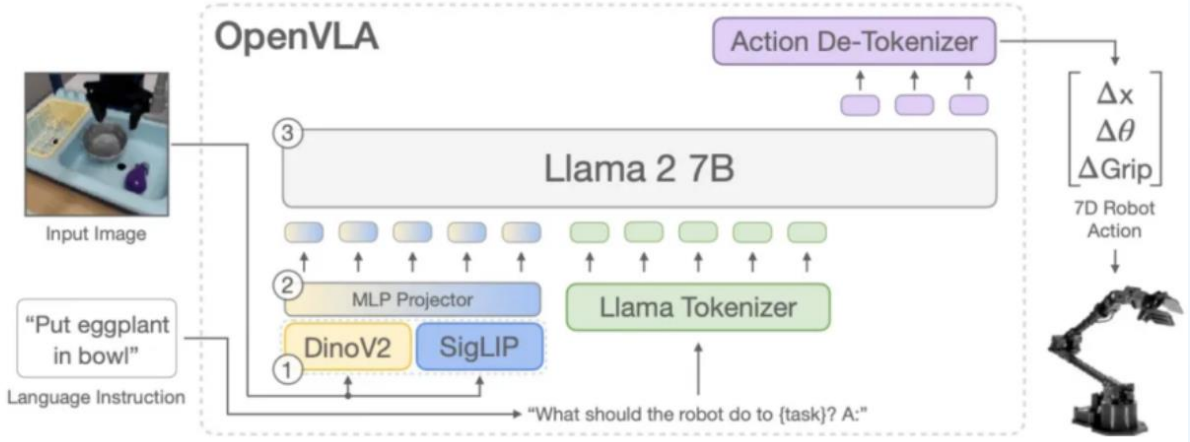
载体的延伸--基于Transformer的大模型技术、深度强化学习控制技术和虚拟仿真技术不仅带来了自动驾驶技术的变革，也深刻影响了机器人领域。

## 2.4.2 具身智能计算系统

如何才能够让具身智能机器人的产品化和规模化落地？需要解决几个关键的技术系统问题：

- 如何搭建具身智能机器人的软件栈；
- 如何满足具身智能机器人的算力；
- 如何获得具身大模型训练所需的数据；

OpenVLA：开源先锋与性能标杆




### 1. 具身智能的软件栈

**A. 端到端软件架构（VLA）**

（1）**优势：**这种架构简化了系统设计，将复杂的任务处理流程整合到一个模型中，从感知到行动一体化处理，可以利用大量的多模态数据进行训练，具有更好的整体性和自适应性；

（2）**劣势：**端到端模型参数量过大，目前只能部署到云端，导致机器人的实时性差，软件运行成本高，除了计算性能瓶颈，在执行任务的精度方面仍然不足；

## 2.4.2 具身智能计算系统

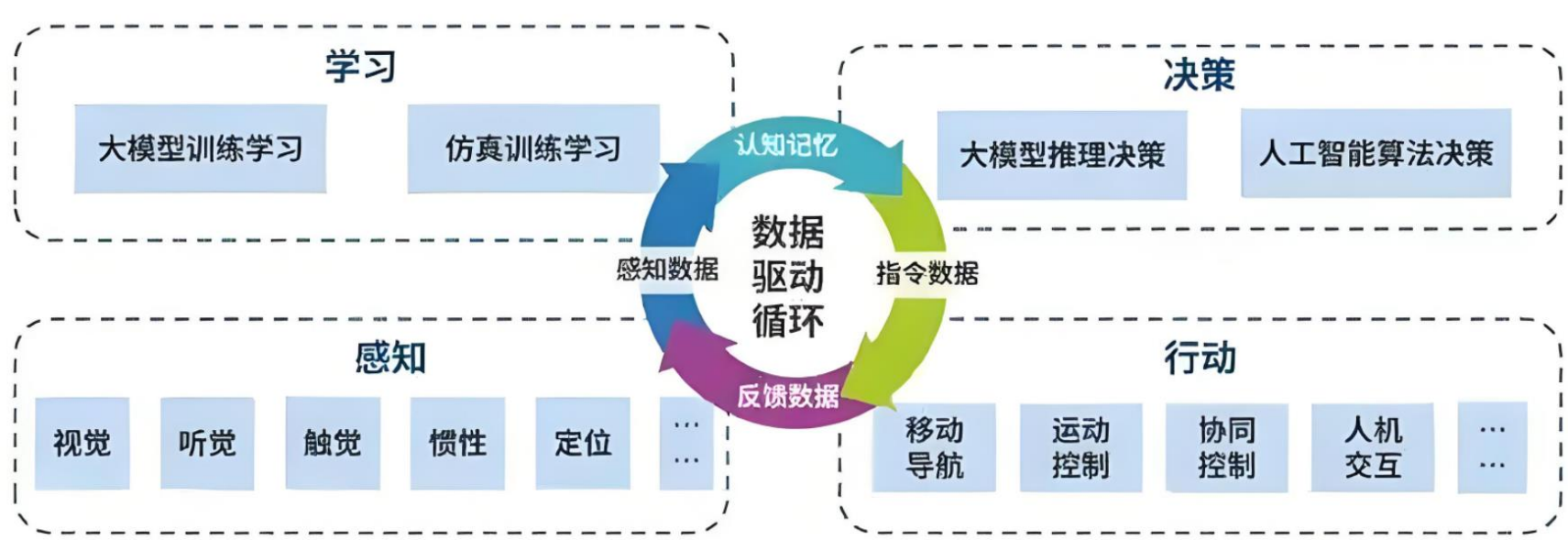
### 1. 具身智能的软件栈

**B. 模块化软件架构**

传统的机器人架构，将机器人的推理、感知、决策和控制分成不同的模块来实现。

（1）**优势：**大模型能够学习和理解大量的数据，并从中提取高层次的知识和信息，从而提高机器人的通用性和鲁棒性，模块化软件架构能够保证机器人计算的精度和实时性；

（2）**劣势：**模块化的设计不可避免地引入人工定义的接口和经验规则，可能会限制机器人的学习能力和执行任务的通用性；



## 2.4.2 具身智能计算系统

### 2. 具身智能机器人硬件计算平台

具身智能应用对计算硬件有以下需求：

（1）高算力--通常基于Transformer架构，这些模型参数量巨大；

（2）高实时性--具身智能机器人需要在实时环境中工作，计算平台必须能够保证低延迟的计算；

（3）低功耗--具身智能机器人通常需要在电池供电的情况下运行，功耗必须受到严格的限制；

（4）高并发性--具身智能机器人需要同时处理来自多个传感器的信息，并执行多个任务；

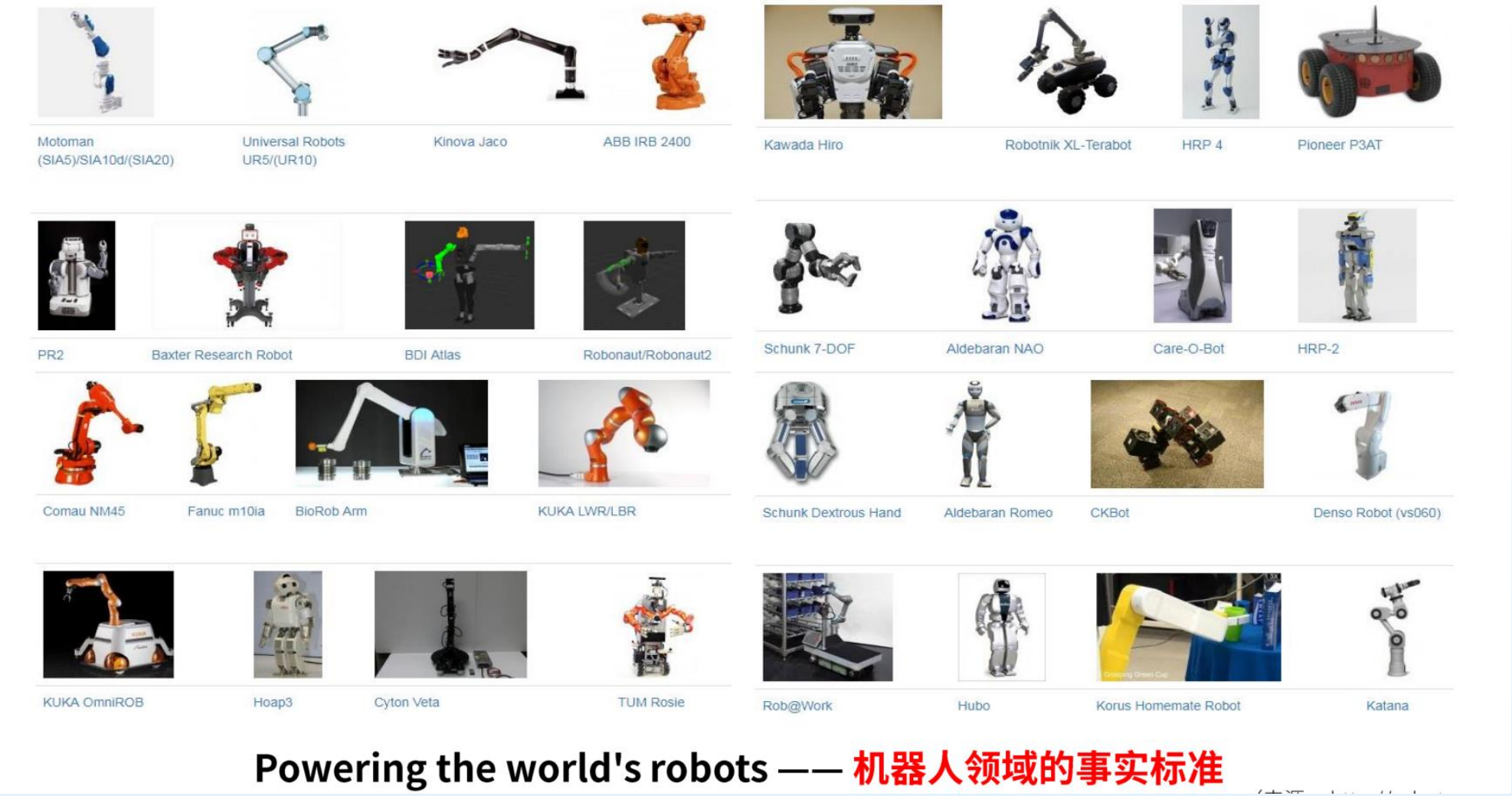
## 2.4.2 具身智能计算系统

### 3. 具身模型训练的数据

- 数据是训练具身模型的原材料，是决定具身模型精度的关键因素。相比于其它人工智能应用，具身智能机器人的本体执行和控制数据需要用实际的机器人执行实际任务并与真实物理世界进行交互，导致数据采集成本高，例如：共有约16位工程师为谷歌RT-1具身智能机器人收集数据，花费了约17个月，用13个机器人收集了13万条数据，涵盖了700多个机器人任务。
- 三种主要的数据来源：互联网上的图文数据、采集机器人传感器和执行器的数据、仿真器合成的虚拟数据

# 2.5 ROS原理与应用

## 什么是ROS?




**Powering the world's robots —— 机器人领域的事实标准**

# 2.6 ROS原理与应用

## 什么是ROS?

**ROS是机器人操作系统（Robot Operating System）的英文缩写。**

- ROS源于斯坦福大学Stanford Artificial Intelligence Robot (STAIR) 和Personal Robotics (PR)的两个项目，随后经过众多开发者的贡献，2007年，柳树车库（Willow Garage）提供了大量资源，用于将斯坦福大学项目中的软件系统进行扩展与完善，同时，在无数研究人员的共同努力下，ROS的核心思想和基本软件包逐渐得到完善。
- ROS是一个应用于机器人上的操作系统，它操作方便、功能强大，特别适用于机器人这种多节点多任务的复杂场景。

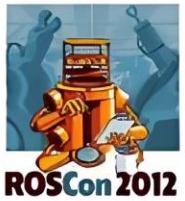
# 2.6 ROS原理与应用

## ROS1与ROS2发展历程

2007
诞生于斯坦福
STAIR项目
Morgan Quigley

Box Turtle

2010
ROS 1.0 发布




2012
第一届ROScon

2014
ROS Indigo发布

2020
ROS Noetic发布

2021
ROS World

2008
Willow Garage接手

Willow Garage

2011
TurtleBot发布

2013
OSRF接管

Open Source Robotics Foundation

2017
ROS 2.0 Ardent发布

2

2020
ROS 2.0 Foxy发布

2022
ROS2 Humble LTS

# 2.5 ROS原理与应用

## 2.5.1 ROS的作用

- ROS（机器人操作系统）是一个开源的机器人软件开发框架，主要用于提供硬件抽象、设备控制、消息传递等功能，常用函数的实现，进程间消息传递，以及包管理。它也提供用于获取、编译、编写、和跨计算机运行代码所需的工具和库函数。支持模块化设计和分布式通信，广泛应用于工业自动化、自动驾驶和服务机器人等领域。
- ROS是一个适用于机器人编程的框架， 这个框架把原本松散的零部件耦合在了一起， 为他们提供了通信架构。 ROS虽然叫做操作系统， 但并非Windows、 Mac那样通常意义的操作系统， 它只是连接了操作系统和你开发的ROS应用程序， 所以它也算是一个中间件， 基于ROS的应用程序之间建立起了沟通的桥梁， 所以也是运行在Linux上的运行环境， 在这个环境上， 机器人的感知、 决策、 控制算法可以更好的组织和运行。

## 2.5.2 ROS系统实现-计算图

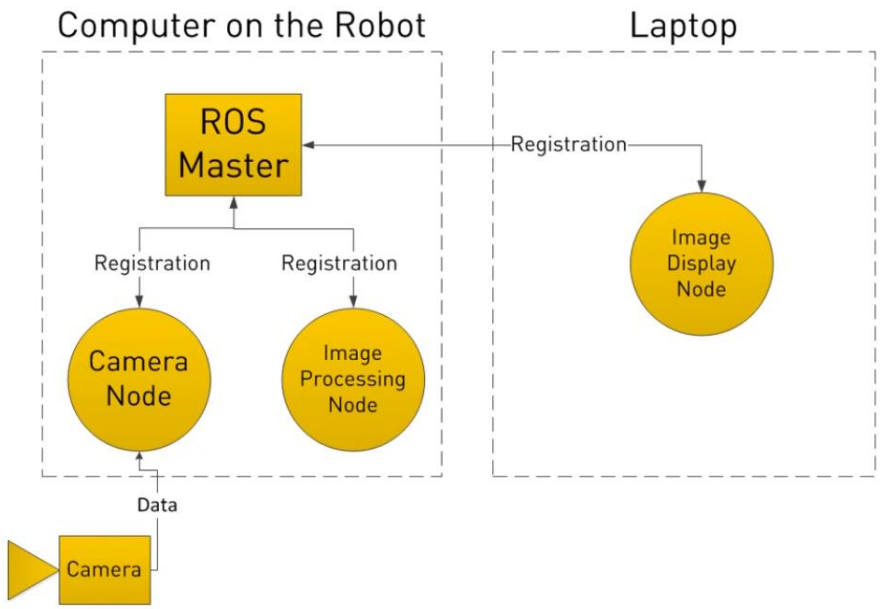
**计算图-ROS处理数据的一种点对点的网络形式。程序运行时，所有进程以及他们所进行的数据处理，将会通过一种点对点的网络形式表现出来。计算图中的重要概念：节点(Node)、节点管理器、消息(message)、话题(topic)、服务(service)。**

■ **节点（Node）** —— 执行单元

- 执行具体任务的进程、独立运行的可执行文件；
- 不同节点可使用不同的编程语言，可分布式运行在不同的主机；
- 节点在系统中的名称必须是唯一的。

■ **节点管理器 （ROS Master）** —— 控制中心

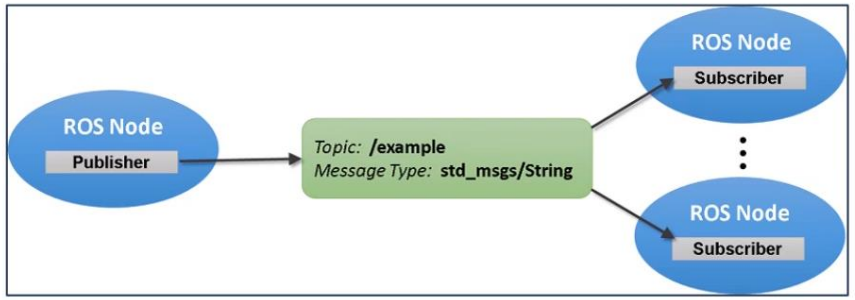
- 为节点提供命名和注册服务；
- 跟踪和记录话题/服务通信，辅助节点相互查找、建立连接；
- 提供参数服务器，节点使用此服务器存储和检索运行时的参数。



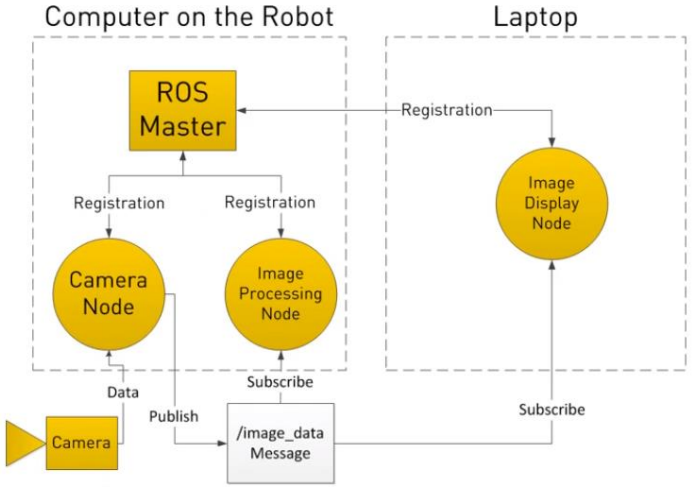
## 2.5.2 ROS系统实现-计算图

**计算图-ROS处理数据的一种点对点的网络形式。程序运行时，所有进程以及他们所进行的数据处理，将会通过一种点对点的网络形式表现出来。计算图中的重要概念：节点(Node)、节点管理器、消息(message)、话题(topic)、服务(service)。**

- **话题（Topic）** —— 异步通信机制
  - 节点间用来传输数据的重要总线；
  - 使用发布/订阅模型，数据由发布者传输到订阅者，同一个话题的订阅者或发布者可以不唯一。
- **消息（Message）** —— 话题数据
  - 具有一定的类型和数据结构，包括ROS提供的标准类型和用户自定义类型；
  - 使用编程语言无关的.msg文件定义，编译过程中生成对应的代码文件。




话题模型（发布/订阅）

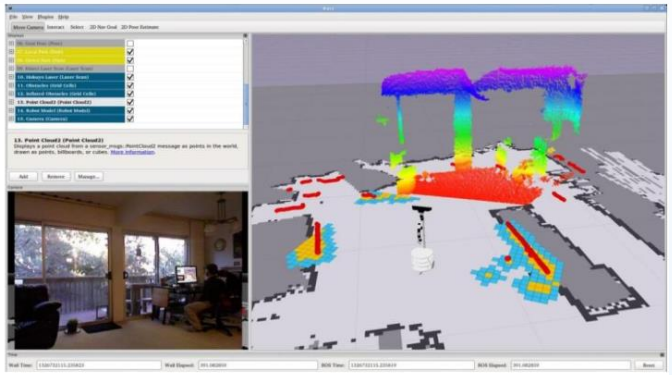


## 2.5.3 ROS的特点

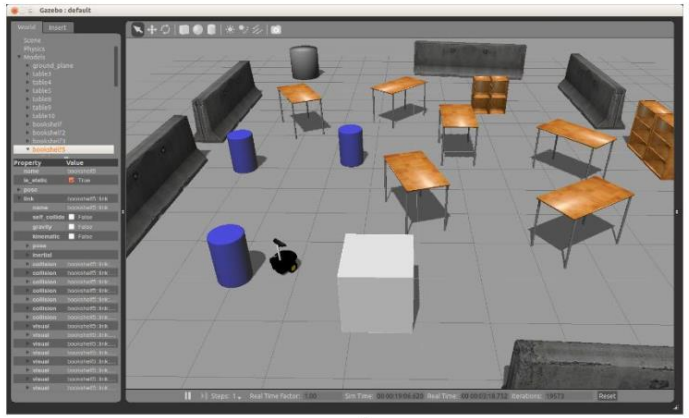
1. ROS支持的编程语言：Python/C++/Java等

2. 组件化工具包丰富

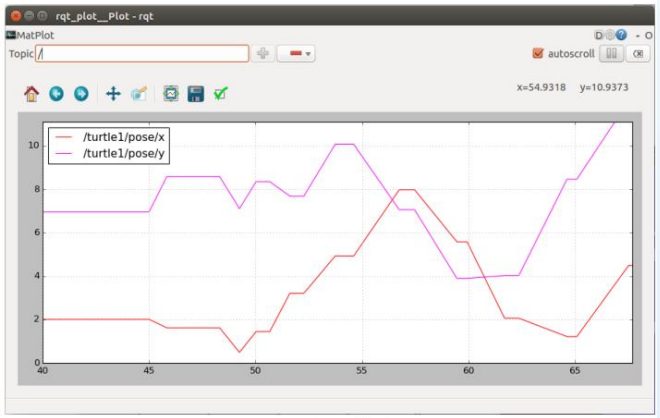
- 3D可视化工具——rviz
- 物理仿真环境——gazebo
- 数据记录工具——rosbag
- Qt工具箱——rqt_*



rviz



gazebo




rqt_plot

## 2.5.4 ROS初次实践-第一个ROS例程（小海龟仿真）

启动ROS Master

```
$ roscore
```

↓

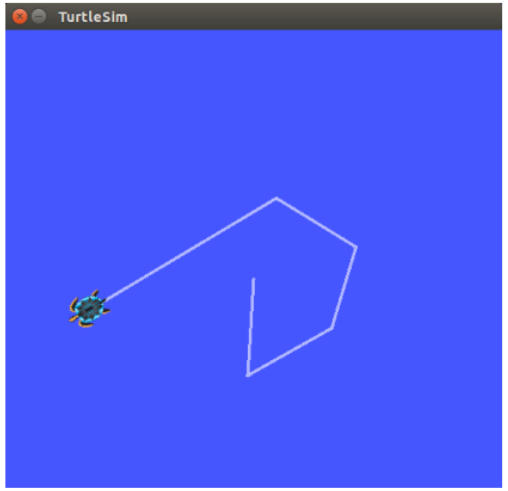
启动小海龟仿真器

```
$ rosrun turtlesim turtlesim_node
```

↓

启动海龟控制节点

```
$ rosrun turtlesim turtle_teleop_key
```




小海龟仿真器界面

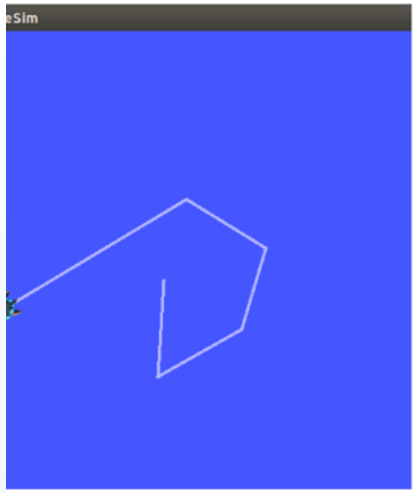
```
→  ~ rosrun turtlesim turtlesim_node
[ INFO] [1520517173.909715844]: Starting turtlesim with node name /turtlesim
[ INFO] [1520517173.920457141]: Spawning turtle [turtle1] at x=[5.544445], y=[5.
544445], theta=[0.000000]
```

启动海龟仿真器节点

```
→  ~ rosrun turtlesim turtle_teleop_key
Reading from keyboard
---------------------------
Use arrow keys to move the turtle.
```

启动海龟控制节点

## 2.5.4 ROS初次实践-第一个ROS例程（小海龟仿真）



小海龟仿真器界面

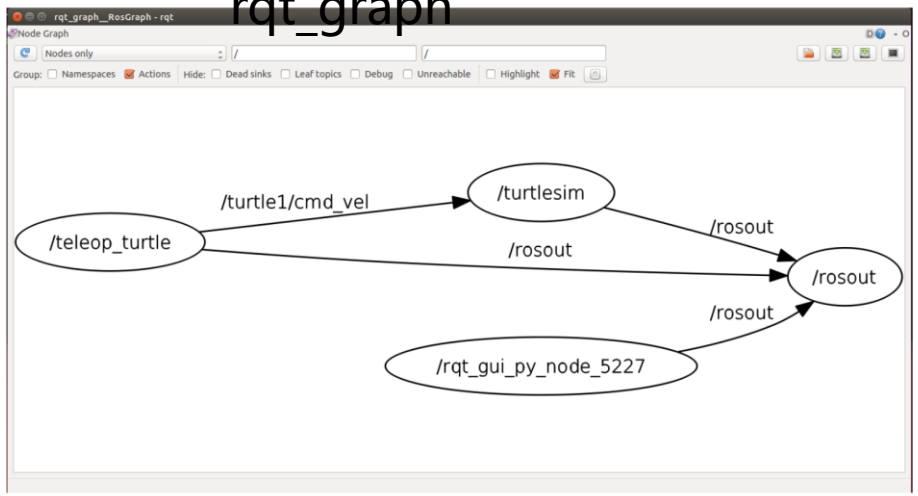
```
➜  ~ rosrun turtlesim turtlesim_node
[ INFO] [1520517173.909715844]: Starting turtlesim wit
[ INFO] [1520517173.920457141]: Spawning turtle [turtl
544445], theta=[0.000000]
```

启动海龟仿真器节点

```
➜  ~ rosrun turtlesim turtle_teleop_key
Reading from keyboard
---------------------------
Use arrow keys to move the turtle.
```

启动海龟控制节点

rosrun rqt_graph
rqt_graph




使用rqt_graph可视化工具查看系统中运行的计算图

## 2.5.5 ROS的安装环境及方法

**课程实验项目准备：**

***Step 1.*** **本机安装VMware workstation虚拟化软件；**

***Step 2.*** **安装Ubuntu20.04 虚拟机；**

***Step 3.*** **安装ROS（ Noetic 版本），具体见文档；**

# 谢 谢！